Software Requirements Specification for Software Engineering: subtitle describing software

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

October 18, 2026

Contents

1 Introduction **1**
- 1.1 Purpose . . . 2
- 1.2 Scope . . . 2
- 1.3 Stakeholders - ITS STAKEHOLDER TIME BOI . . . 3
- 1.4 Definitions and Acronyms . . . 3
- 1.5 Characteristics of Intended Reader . . . 4

2 System Overview **4**
- 2.1 Problem Statement . . . 4
- 2.2 Product Vision . . . 5
- 2.3 System Context . . . 5
- 2.4 User Characteristics . . . 7
- 2.5 System Constraints . . . 7

3 Stakeholder Requirements **8**

4 User Personas **9**
- 4.1 Persona 1 . . . 9
- 4.2 Persona 2 . . . 9
- 4.3 Persona 3 . . . 10

5 Use Cases **11**
- 5.1 UC1: Example Use Case . . . 11

6 Functional Requirements **11**

7 Data Requirements **13**
- 7.1 Major Data Entities . . . 13
- 7.2 Data Dictionary . . . 13
- 7.3 Privacy and Data Retention . . . 13

8 Non-Functional Requirements **14**

9 AI and Machine Learning Requirements **15**
- 9.1 Model Inputs . . . 15
- 9.2 Model Outputs . . . 15
- 9.3 Evaluation Metrics . . . 15
- 9.4 Known Limitations . . . 15

10 Requirements Traceability Matrices **15**

11 Rationale **16**

12 Likely Changes **16**

13 Unlikely Changes **16**

14 Appendices **17**

14.1 Appendix A: Additional Materials 17
14.2 Appendix B: Development Plan 17
14.3 Appendix C: Values of Auxiliary Constants 17
14.4 Appendix D: Generative AI Disclosure 17
14.5 Appendix E: Reflection 17

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[All capstone SRS documents should have a reflection appendix. —TPLT]

[This SRS template is based on the IEEE template, with input from co-pilot and S. Smith. It will get you started. You should not modify the section headings, without first discussing the change with the course instructor. Modification means you are not following the template, which loses some of the advantage of a template, especially standardization. —TPLT]

[Feel free to change the appearance of the report by modifying the LaTeX commands. —TPLT]

[This template document assumes that a single program is being documented. If you are documenting a family of models, you should start with a commonality analysis. —TPLT]

[The SRS is not generally written, or read, sequentially. The SRS is a reference document. It is generally read in an ad hoc order, as the need arises. —TPLT]

[Guiding principles for the SRS document:

- Do not repeat the same information at the same abstraction level. If information is repeated, the repetition should be at a different abstraction level.

—TPLT]

[The template description comments should be disabled before submitting this document for grading. —TPLT]

[You can borrow any wording from the text given in the template. It is part of the template, and not considered an instance of academic integrity. Of course, you need to cite the source of the template. —TPLT]

[When the documentation is done, it should be possible to trace back to the source of every piece of information. Some information will come from external sources, like terminology. —TPLT]

[An SRS document should have the following qualities: unambiguous, consistent, complete, validatable, abstract and traceable. —TPLT]

[The overall goal of the SRS is that someone can learn, understand and verify the captured domain knowledge. They should not have to trust the authors of the SRS on any statements. They should be able to independently verify/derive every statement made. —TPLT]

1 Introduction

[Explain what system is being specified. State purpose, users, and project goals. —TPLT]

[Introduce the name of your software as Software Engineering. Using Software Engineering as a command lets you change the name if necessary and have the change propagate to all documents. —TPLT]

[Include a reference to the template this work is based on. **?** justifies the SRS structure and **?** provides the broader life-cycle context that underlines modern requirements engineering practice. —TPLT]

1.1 Purpose

The CAS (Computing and Software) Department currently manages course outlines and assessment schedules on a course-by-course basis. Course instructors set their deadlines independently, so students in the same program and year can face busy weeks of midterms and deadlines with no department-level visibility into them. CAS administrative staff must also manually check each outline against McMaster course outline policy and the Canadian Engineering Accreditation Board (CEAB) requirements, and follow up with instructors when information is missing.

The CAS Deadline Management System is intended to make assessment dates visible across courses and to identify policy and accreditation gaps early. It reduces the manual effort of checking outlines against these requirements and helps avoid weeks with heavily clustered assessments. Instructors and administrators remain in the loop and responsible for every decision. The system only flags issues and makes suggestions, and never changes a course outline or assessment date on its own.

1.2 Scope

The CAS Deadline Management System aims to improve course outline review and assessment scheduling within the CAS Department at McMaster University. The system addresses the lack of department-wide visibility into assessment dates and the manual effort required to verify course outlines against university policies and CEAB requirements. The system will allow CAS administrators to upload course outlines, extract and store relevant course and assessment information, identify missing information and potential policy or accreditation gaps, and visualize assessment schedules across courses. It will also identify potential assessment conflicts and suggest alternative dates to help distribute assessments more evenly throughout the academic term. The system will support reviewing and exporting extracted information to facilitate administrative coordination.

The primary users are CAS administrators and course instructors, while students may benefit indirectly from improved assessment scheduling. The system is intended to reduce administrative workload, improve policy compliance, and support more balanced assessment schedules.

The system will provide recommendations for review rather than make decisions independently. It will not automatically modify course outlines, change assessment dates, or override instructor decisions. Instructors retain authority over their course schedules, and administrators remain responsible for reviewing policy compliance and accreditation requirements. The system's recommendations will be based on the available course outline data, timetable information, and defined policies; it will not guarantee that all scheduling conflicts or compliance issues are identified.

1.3 Stakeholders - ITS STAKEHOLDER TIME BOI

The CAS Midterm and Assignment Optimization System is intended to support the Computing and Software (CAS) department at McMaster University in reviewing course outlines provided by instructors and coordinating assessment schedules. The stakeholders include individuals who directly use the system, individuals affected by its recommendations, and organizations responsible for ensuring policy requirements and accreditation.

Primary Stakeholders:

- CAS Administration Team (Customer)
- CAS Department Professors and Instructors (End Users)
- Capstone Project Team 19 (Development Team)

Secondary Stakeholders:

- Software Engineering Students (Indirectly affected by assessment scheduling recommendations)
- McMaster University (Affected by the system's support for course coordination and policy compliance)

Tertiary Stakeholders:

- Canadian Engineering Accreditation Board (CEAB)
- Future CAS Administrators and Instructors

1.4 Definitions and Acronyms

Term	Definition
CEAB	Canadian Engineering Accreditation Board, the organization responsible for accrediting undergraduate engineering programs in Canada.
LLM	Large Language Model, an AI model trained on large amounts of text to understand and generate human language.
SRS	Software Requirements Specification, a document that describes a software system's requirements, including its expected functionality and constraints.

Table 1: Definitions and Acronyms

1.5 Characteristics of Intended Reader

The intended readers of this Software Requirements Specification include project supervisors, CAS administrators, software developers, testers, and future maintainers. Technical readers should have a foundational understanding of software development, web applications, databases, and software testing. Familiarity with PDF processing and large language models (LLMs) is beneficial but not required. CAS administrators and project supervisors should be familiar with academic scheduling, course outlines, and relevant university policies, including CEAB requirements. This document is intended to communicate system requirements clearly to both technical and non-technical readers.

2 System Overview

This section provides general information about the system. It identifies the problem, the vision, and the interfaces between the system and its environment. This section also describes user characteristics and lists the system constraints.

2.1 Problem Statement

The CAS Department at McMaster University currently manages course outlines and assessment schedules independently across courses. Without a centralized view of assessment dates, students in the same program and academic year may face multiple midterms and assignment deadlines within short periods. CAS administrative staff must also manually review course outlines for compliance with university policies and CEAB requirements. Missing or incomplete information, such as midterm dates listed as "TBD," creates additional work and requires repeated communication with instructors.

These challenges contribute to uneven student workloads, increased academic stress, and significant administrative overhead. Furthermore, instructors retain authority over their course schedules, limiting administrators' ability to directly resolve scheduling conflicts.

The CAS Deadline Management System aims to improve this process by centralizing assessment information, identifying potential scheduling conflicts and policy compliance gaps, and suggesting alternative assessment dates. This will help administrators coordinate schedules more efficiently, reduce manual review efforts, and support more balanced student workloads while preserving instructors' authority over their courses.

2.2 Product Vision

The CAS Deadline Management System will provide the CAS Department with a trusted, department-wide foundation for collaborative assessment planning. It will turn course-outline information into a consistent, reviewable source of scheduling knowledge that helps instructors and administrators make informed decisions together. The system is intended to improve coordination and transparency over time, while keeping final responsibility for course schedules and policy interpretation with the appropriate human decision-makers.

2.3 System Context

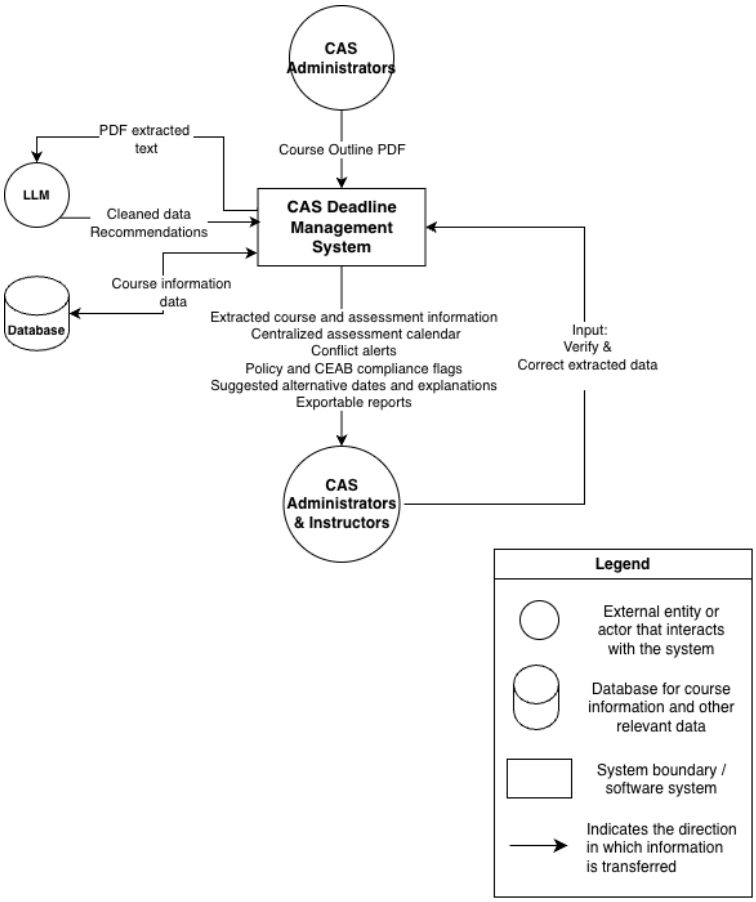


Figure 1: System Context

The CAS Deadline Management System is a web-based application intended to support course outline review, assessment scheduling, and policy compliance within the CAS Department at McMaster University. CAS administrators and course instructors interact with the system to upload course outlines, review extracted information, view assessment schedules, and evaluate potential scheduling conflicts and compliance issues. Figure 1 summarizes

these interactions by showing the system boundary and its relationships with instructors, CAS administrators, the LLM, and the database.

The system processes course outlines and relevant academic scheduling information to extract assessment details, identify missing or potentially non-compliant information, and recommend alternative assessment dates. Results are presented to users for review, while instructors and authorized administrators retain responsibility for final scheduling and compliance decisions.

The system is intended for academic administration and planning rather than mission-critical or safety-critical applications. Since extracted information and automated recommendations may be incomplete or inaccurate, users must review flagged issues and verify important results before making decisions.

- **User Responsibilities:**
  - Provide course outlines and relevant academic scheduling information.
  - Ensure submitted documents are appropriate for review and that the information provided is sufficiently complete and accurate.
  - Review extracted course and assessment information and correct errors where supported.
  - Evaluate flagged scheduling conflicts, missing information, and potential policy compliance issues.
  - Decide whether to accept or reject suggested assessment dates and determine any final changes to course schedules.
- **Software Engineering Responsibilities:**
  - Accept and process supported course outline documents and scheduling information.
  - Extract and organize relevant course and assessment details for review.
  - Validate extracted data for missing, inconsistent, or incorrectly formatted values where possible.
  - Present assessment schedules and identify potential workload conflicts.
  - Flag potential gaps in course outline requirements and applicable accreditation criteria.
  - Generate alternative assessment date recommendations with supporting explanations.
  - Present results and warnings to users without automatically changing course outlines or assessment schedules.

- **External System Responsibilities:**
  - **LLM:** Process course outline content to support information extraction and, where applicable, the generation of explanations for scheduling recommendations.
  - **Database:** Store and retrieve course information, extracted assessment details, review results, and scheduling information.

2.4 User Characteristics

The primary users of the CAS Deadline Management System are CAS administrators and course instructors at McMaster University. Users are expected to have basic computer skills, including navigating web applications and viewing course information. CAS administrators should be familiar with course outline reviews, academic scheduling, and applicable university policies. Course instructors should have knowledge of their own course outlines and assessment schedules. No programming or machine learning knowledge is required. Users should be able to review extracted information, interpret flagged issues and scheduling recommendations, and make informed decisions regarding their courses or administrative responsibilities.

2.5 System Constraints

The CAS Deadline Management System must operate within the technical and institutional constraints of McMaster University. The following constraints apply to the system:

- **Deployment Environment:** The system must be deployable within the computing resources made available by McMaster University. The availability and capacity of these resources may limit the models and processing workloads the system can support.
- **Data Sources:** The system must process course outlines provided in PDF format. Variations in document structure, formatting, and scan quality may affect the accuracy of information extraction.
- **Language Model Cost and Hosting:** The system must use an LLM that is available at no cost to the project. GitHub Copilot should be preferred where it satisfies the system's technical and institutional requirements. Otherwise, the system should prioritize locally hosted, open-weight language models to minimize reliance on external services. Model selection must account for available computing resources and applicable licensing restrictions.
- **Human Decision-Making:** The system must not automatically modify course outlines or assessment dates. All scheduling changes and final compliance decisions remain the responsibility of instructors and authorized administrators.

- **Policy and Accreditation Requirements:** Compliance checks must be based on documented McMaster University policies and applicable CEAB requirements. The system must support human review of flagged issues rather than treating automated results as definitive determinations.

3 Stakeholder Requirements

SR1 Centralized Course Outline Submission: CAS administrators need a centralized location to collect and review course outlines.

SR2 Course Information Extraction: CAS administrators need relevant information extracted from submitted course outlines, including course details, assignments and projects, marking schemes, and midterm dates where available.

SR3 McMaster Policy Review: CAS administrators need support in identifying potential failures of course outlines to comply with McMaster University and CAS department policies.

SR4 CEAB Accreditation Review: CAS administrators need support in reviewing course outlines against CEAB accreditation requirements, including evaluating how each course addresses relevant CEAB graduate attributes.

SR5 Assessment Schedule Analysis: Professors/instructors and CAS administrators need to identify periods with high concentrations of assessment and midterm deadlines across relevant CAS department courses and reassess the schedule when dates are updated.

SR6 Scheduling Recommendations: Professors/instructors need recommendations for alternative midterm and assessment dates that may reduce deadline bottlenecks for students, while being able to make the final decision.

SR7 Instructor Decision-Making: Professors/instructors need to retain authority over whether to accept or reject scheduling recommendations.

SR8 Handling Incomplete Information: Professors/instructors and CAS administrators need to be informed when missing or unreadable course outline information may affect the reliability of compliance reviews or scheduling recommendations.

SR9 Authorized Access: CAS administrators and professors/instructors need access to the information and functions relevant to their roles, while assuring sensitive information is protected and restricted to authorized users.

SR10 Shared Assessment Calendar: Professors/instructors and CAS administrators need access to a shared calendar displaying relevant course assessment dates.

SR11 Exam and Restricted-Period Conflicts: Professors/instructors and CAS administrators need to identify potential conflicts between assessments, exam dates, and applicable restricted periods.

4 User Personas

4.1 Persona 1

- Name: Bob M. Builder (CAS Administrator)
- Description: Bob is responsible for collecting and reviewing CAS course outlines to help ensure they align with McMaster University and CAS department policies, as well as CEAB accreditation requirements. Bob needs an efficient way to carry out his administrative duties while reducing the amount of redundant back and forth communication with instructors to ensure that outlines are aligned with the required policies and accreditation requirements.
- Goals:
  - To collect and review course outlines in one centralized location.
  - To identify missing information and potential policy or accreditation issues.
  - To reduce the time spent manually reviewing multiple course outlines.
  - To reduce unproductive communication time with course instructors.
- Frustrations:
  - Course outlines may use different formats or organize information differently.
  - Manually locating relevant information across many documents can be time-consuming.
  - Missing or unclear information can make it difficult to assess compliance.
- Technical Ability: Moderate (experience with administrative software, document management tools, spreadsheets)

4.2 Persona 2

- Name: Dora Sanchez Explora (Software Engineering Professor)
- Description: Dora teaches Software Engineering Design courses and plans assignments and two midterms for the semester. She wants to provide students with a workload that is easy to manage and deadlines that allow them to learn and adequately prepare for their midterms without overwhelming them in their other courses.

- Goals:
  - To submit and maintain tentative assessment dates efficiently.
  - To identify potential conflicts with assessments in other courses.
  - To receive practical recommendations for distributing assessment deadlines more evenly.
  - To retain final authority over assessment dates for her course.
- Frustrations:
  - Coordinating assessment dates across courses can be difficult.
  - Several courses may schedule major assessments around the same time.
  - Changing an assessment date may create conflicts with other course activities or require additional coordination.
- Technical Ability: Moderate (experience with administrative management systems, online course and university tools)

4.3 Persona 3

- Name: Homer Simpson (4th Year Software Engineering Student)
- Description: Homer is an undergraduate Software Engineering student taking six courses for the Fall term. He is feeling overwhelmed by the workload and is struggling to manage his time between getting his work done and studying for his upcoming back-to-back midterms.
- Goals:
  - To have a more balanced distribution of assessments throughout the term.
  - To avoid periods when multiple major deadlines occur close together.
  - To have sufficient time to prepare for major assessments across different courses.
- Frustrations:
  - Multiple assignments and midterms may fall within the same few days.
  - Heavy assessment periods can make it difficult to allocate study/work time effectively.
  - Assessment dates across different courses may be difficult to coordinate.
- Technical Ability: High (experience with online learning platforms, digital calendars, student productivity tools and software)

5 Use Cases

[Use cases are a way to describe the interactions between users and the system. They are typically written in natural language and describe the steps that a user takes to achieve a specific goal. Use cases can be used to identify functional requirements and to create test cases. —TPLT] [Typically 8-15 substantial use cases. —TPLT]

5.1 UC1: Example Use Case

Primary Actor

TODO

Preconditions

- TODO

Main Success Scenario

1. TODO
2. TODO
3. TODO

Alternative Flows

- TODO

Postconditions

- TODO

6 Functional Requirements

The system shall support the following capabilities, grouped by feature area.

Outline Intake and Extraction

FR1: The system shall accept course outlines in PDF format submitted by a CAS administrator or professor.

FR2: The system shall extract each assessment's name, type, date, and weight from a course outline.

FR3: The system shall extract the learning outcomes and graduate attribute numbers listed in a course outline.

FR4: The system shall flag any extracted field that is missing, including a date that is marked TBD.

FR5: The system shall require a user to review and confirm extracted data before it is used for the calendar, conflict detection, or compliance checking.

FR6: The system shall allow an instructor to enter and update tentative assessment dates for their own courses.

Calendar and Conflict Detection

FR7: The system shall display confirmed and tentative assessment dates in a shared calendar that can be filtered by course, year, and program, with tentative dates marked as such.

FR8: The system shall identify weeks in which a student cohort has more assessments than a configurable threshold.

FR9: The system shall identify assessments scheduled in periods restricted by McMaster policy, such as final exam periods.

FR10: The system shall re-run conflict detection for the affected courses and cohorts whenever an assessment date is added or changed.

Scheduling Suggestions

FR11: The system shall, on explicit request by the instructor, suggest alternative dates, for assessments involved in a detected conflict or listed as TBD.

FR12: The system shall not change any assessment date without the explicit approval of the responsible instructor.

Compliance Checking

FR13: The system shall check each outline against the McMaster course outline policy rules, including required sections and assessment weights totalling 100%, and report each finding.

FR14: The system shall compare the graduate attribute numbers in an outline with the master course sheet and flag any that are missing or inconsistent.

Access and Export

FR15: The system shall require users to log in before they can view or change any data.

FR16: The system shall allow administrators to manage data for all courses, and shall allow instructors to change data only for their own courses.

FR17: The system shall allow an administrator to export extracted assessment data as a CSV file.

7 Data Requirements

vanessa

[The Data Requirements section describes what information the system must manage. It includes the major data entities, their attributes, and the relationships between them. It also includes a data dictionary that defines the meaning of each attribute. —TPLT]

7.1 Major Data Entities

[What information must exist for the system to accomplish its goals? —TPLT]

[For each entity, provide a short explanation. —TPLT]

- User
- Song
- Session
- Report

7.2 Data Dictionary

[For each entity, define all important attributes. —TPLT]

Attribute	Type	Description
user_id	UUID	Unique identifier
email	String	User email address

Table 2: Data Dictionary

7.3 Privacy and Data Retention

[Discuss storage, retention, deletion, privacy controls, consent, encryption, and compliance requirements. —TPLT]

TODO

8 Non-Functional Requirements

[You may have to add other NFRs. —TPLT]

[List your nonfunctional requirements. You may consider using a fit criterion to make them verifiable. —TPLT]

[The goal is for the nonfunctional requirements to be unambiguous, abstract and verifiable. This isn't easy to show succinctly, so a good strategy may be to give a "high level" view of the requirement, but allow for the details to be covered in the Verification and Validation document. —TPLT]

[An absolute requirement on a quality of the system is rarely needed. For instance, an accuracy of 0.0101 % is likely fine, even if the requirement is for 0.01 % accuracy. Therefore, the emphasis will often be more on describing now well the quality is achieved, through experimentation, and possibly theory, rather than meeting some bar that was defined a priori. —TPLT]

[You do not need an entry for correctness in your NFRs. The purpose of the SRS is to record the requirements that need to be satisfied for correctness. Any statement of correctness would just be redundant. Rather than discuss correctness, you can characterize how far away from the correct (true) solution you are allowed to be. This is discussed under accuracy. —TPLT]

NFR1: **Accuracy** [Characterize the accuracy by giving the context/use for the software. Maybe something like, "The accuracy of the computed solutions should meet the level needed for <engineering or scientific application>. The level of accuracy achieved by Software Engineering shall be described following the procedure given in Section X of the Verification and Validation Plan." If you do this provide a link to the VnV plan. —TPLT]

NFR2: **Usability** [Characterize the usability by giving the context/use for the software. You should likely reference the user characteristics section. The level of usability achieved by the software shall be described following the procedure given in Section X of the Verification and Validation Plan. Include a link to the VnV plan. —TPLT]

NFR3: **Maintainability** [The effort required to make any of the likely changes listed for Software Engineering should be less than FRACTION of the original development time. FRACTION is then a symbolic constant that can be defined at the end of the report. —TPLT]

NFR4: **Portability** [This NFR is easier to write than the others. The systems that Software Engineering should run on should be listed here. When possible the specific versions of the potential operating environments should be given. To make the NFR verifiable a statement could be made that the tests from a given section of the VnV plan can be successfully run on all of the possible operating environments. —TPLT]

- Other NFRs that might be discussed include verifiability, understandability, reusability, performance, reliability and availability, security, and accessibility. You may have to

add other NFRs.

9 AI and Machine Learning Requirements

vanessa

[Refer to Dr. Mosser's guest lecture on AI requirements. —TPLT]

[Include only when applicable. —TPLT]

9.1 Model Inputs

TODO

9.2 Model Outputs

TODO

9.3 Evaluation Metrics

[Accuracy, Precision, Recall, F1, BLEU, WER, etc. —TPLT]

TODO

9.4 Known Limitations

[Explicitly document expected failure modes. —TPLT]

TODO

10 Requirements Traceability Matrices

[The Requirements Traceability Matrix (RTM) is a table that shows the relationship between different types of requirements. It is used to ensure that all requirements are traceable and that no requirements are missed. The typical flow is: Stakeholder Need -¿ Use Case -¿ Requirement —TPLT]

[There is also a traceability matrix between functional and nonfunctional requirements. The purpose of some FRs is to satisfy NFRs. Therefore, it is important to show the relationship between the two. —TPLT]

[Further traceability to test cases and modules will be given in other documents. —TPLT]

Stakeholder Req	Use Case	Functional Req
SR??	UC1	FR??

Table 3: Traceability Between Stakeholder Requirements, Use Cases, and Functional Requirements

Functional Req	Nonfunctional Req
FR??	NFR1

Table 4: Traceability Between Functional and Nonfunctional Requirements

11 Rationale

[Provide a rationale for the decisions made in the documentation. Rationale should be provided for scope decisions, functional requirements, nonfunctioanl requirements, data requirements, and AI requirements. You don't need a rationale for everything, ust the major decisions that are not obvious. You should have traceability to the entities in the document that you are referencing. —TPLT]

12 Likely Changes

LC1: [Give the likely changes, with traceability to the entity that is likely to change. —TPLT]

13 Unlikely Changes

LC2: [Give the unlikely changes. The design can assume that the changes listed will not occur. —TPLT]

14 Appendices

14.1 Appendix A: Additional Materials

TODO

14.2 Appendix B: Development Plan

[This section is optional. It is used to explain the plan for developing the software. In particular, this section gives a list of the order in which the requirements will be implemented. In the context of a course this is where you can indicate which requirements will be implemented as part of the course, and which will be "faked" as future work. This section can be organized as a prioritized list of requirements, or it could show the requirements that will be implemented for "phase 1", "phase 2", etc. —TPLT]

14.3 Appendix C: Values of Auxiliary Constants

[Show the values of the symbolic parameters introduced in the report. —TPLT]

[The definition of the requirements will likely call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance. —TPLT]

[The value of FRACTION, for the Maintainability NFR would be given here. —TPLT]

TODO

14.4 Appendix D: Generative AI Disclosure

[Provide full transparency on how generative AI was used in the creation of this document. This includes any text, figures, or tables that were generated by AI. If any skills were created for AI agents to review the documentation, those should be disclosed here. —TPLT]

14.5 Appendix E: Reflection

[This isn't part of the SRS, but it is part of the course. —TPLT]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?